机器学习知识点

1. 机器学习基础

- **任务分类**：
  - **分类 (Classification)**：目标是预测离散标签（如：猫/狗、晴天/雨天）。
  - **回归 (Regression)**：目标是预测连续数值（如：具体的气温、房价）。
- **数据集划分**：
  - **训练集 (Training Set)**：用于模型真正“学习”和调整模型参数（如权重 W、偏置 b）。
  - **验证集 (Validation Set)**：用于模型评估和调整模型超参数（如学习率 η、K 值）。
  - **测试集 (Test Set)**：评估模型的泛化能力。
- **模型超参数**：学习率、正则化系数等属于手动设置或交叉验证调优的“超参数”。超参数调优的常用方法包括网格搜索、随机搜索及贝叶斯优化。
- **模型参数**：网络权重矩阵属于模型学习到的“参数”。每个神经元都有不同的权重和偏差值，权重和偏差都是神经网络的模型参数。
- **图像分类的挑战**：面临语义鸿沟、背景杂波（干扰）、视角变化、照明、变形、遮挡、类内变化等诸多问题。
- **算法三大支柱**：几乎所有的机器学习算法都由以下三部分组成：
  1. **表示 (Representation)**：设定候选函数空间（如线性模型、神经网络）。
  2. **评价 (Evaluation)**：设定衡量标准（如损失函数、准确率）。
  3. **最优化 (Optimization)**：寻找最优模型的搜索方法（如梯度下降）。

2. K 近邻算法(KNN)

- **核心机制**：KNN 是典型的“懒惰学习”，无显式训练阶段，仅存储数据；预测时直接计算样本间的距离。
- **距离度量**：
  - L1 (曼哈顿)距离：$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$。
  - L2 (欧氏)距离：$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$。
- **决策规则**：在分类任务中，KNN 从 K 个最近点中进行**多数投票**来决定最终标签。
- **参数敏感性与问题**：KNN 超参数（K值、距离度量）是人工设定的，而不是自动学习的，非常依赖于具体问题。模型预测结果高度依赖 K 值的选择（K 值通常选奇数以避免平局）。
- **交叉验证**：当数据量较少时，常使用 K 折交叉验证来最大化数据利用率。KNN 中的 N 折交叉验证：将数据分成 N 份，尝试每一份依次作为验证集，并计算结果的平均值。

3. 损失函数

- **交叉熵损失**：衡量预测概率与真实标签的差异，本质是惩罚模型对正确标签的“不自信”，强调正确分类的概率越大越好。
  - 公式为 $L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$。初始化时所有 s 近似相等，这时 loss 值是 $\log(C)$。
  - **极值范围**：最小值为 0（概率趋近 1），最大值为 $+\infty$。
- **SVM 损失**：核心在于“安全间距”，强调正确类别的得分必须比错误类别高出一定的安全间距，超过即损失为 0。
  - 公式为：$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$。
  - 数据集上的总损失为：$L = \frac{1}{N} \sum_{i=1}^{N} L_i$。初始化时所有 $s \approx 0$，这时 loss 值是 $N - 1$。

- **极值范围**：最小值为 0，最大值为 $+\infty$。
- **含正则化项的总损失**：引入正则化后的损失公式表达为：$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i + \lambda R(W)$。

4. 正则化

- **正则化的目的**：防止过拟合，抑制对训练数据中噪音的拟合；在权重上表达偏好；提高泛化能力（使模型变简单，以便在测试数据上更稳定）。
- **L1 正则化**：在原始损失上增加权重绝对值之和的惩罚项，倾向于产生稀疏权重矩阵，自带特征选择功能。具体公式为 $R_1(W) = \sum_k \sum_l |W_{k,l}|$。
- **L2 正则化 (权重衰减)**：在原始损失上增加权重平方和的惩罚项，限制权重数值过大以防止过拟合，倾向于让所有权重变小但非零，使模型更平滑。具体公式为 $R(W) = \sum_k \sum_l W_{k,l}^2$。注意：偏置项通常不参与正则化约束。梯度变化为：新梯度 = 原梯度 + λw_i。
- **弹性网络**：结合 L1 与 L2 正则化，公式为 $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$。

5. 神经网络与反向传播

- **线性分类器的局限性**
  - **决策边界**：只能绘制线性决策边界，无法妥善处理复杂的非线性数据分布（如环形、多簇分布）。
  - **模板限制**：每个类别只能学习单一模板，表达能力有限。
- **人工神经元结构**
  - **线性计算**：接收输入特征，加权求和并加上偏差，得到中间值 $z = Wx + b$。
  - **非线性激活**：将中间值 z 传入**激活函数**，引入非线性表达能力，得到最终输出 a。
- **神经网络架构**
  - **基本组成**：输入层 (Input Layer) $\rightarrow$ 隐藏层 (Hidden Layers) $\rightarrow$ 输出层 (Output Layer)。
  - **网络深度**：神经网络的深度 = 隐藏层 + 输出层。计算网络层数时**不包含输入层**。例如，“1个输入层 + 2个隐藏层 + 1个输出层”被称为**3层神经网络**。
- **学习目标**：给定网络结构后，通过算法找到一组最优参数（权重 W 和偏差 b），使得在所有训练数据上的**总体损失函数 L 达到最小**。
- 梯度：梯度指向损失函数上升最快的方向，为降低损失，参数需向梯度的反方向更新。
  - $\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_i}$
  - $\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial A} \odot \frac{\partial A}{\partial Z}$
  - $\frac{\partial L}{\partial W} = \frac{\partial L}{\partial Z} X^T$
  - $\frac{\partial L}{\partial X} = W^T \frac{\partial L}{\partial Z}$
- **梯度检查**：在实践中，总是使用解析梯度，但是用数值梯度检查实现，这被称为梯度检查。
- **反向传播更新参数**：沿损失函数对参数梯度的负方向更新。公式为 $w_i^{new} = w_i^{old} - \eta \frac{\partial L}{\partial w}$ 或矩阵形式 $W^{(new)} = W^{(old)} - \eta \frac{\partial L}{\partial W}$，其中 η 为学习率。
- **小批量随机梯度下降**
  - **全批量痛点**：当训练集样本量 N 极大时，每走一步都要计算全部样本的总损失和梯度，极其耗费计算资源且缓慢。
  - **解决方案**：每次只从训练集中随机抽取一小批样本（通常大小为 32、64、128 或 256 等，称为批大小 Batch Size），用这批样本的梯度来近似代表整体的梯度。
  - **优势**：大幅提升了单次计算的速度和参数更新的频率，是目前深度学习最常用的优化方式。

6. 卷积神经网络(CNN)

- **为什么 CNN 适用于图像识别**：

- 一些模式比整张图片小得多；一个神经元不需要看到整个图像去发现模式，通过较少的参数连接到小区域（例如“喙”探测器识别出一张鸟的图片中的鸟嘴即可识别出鸟）。
- 同样的模式可能出现在图像的不同区域（如左上的鸟喙和中间的鸟喙），它们可以使用相同的一组参数（share 参数）。
- 对图像进行二次采样不会改变图像中的物体。

(1) 卷积神经网络中的层结构

- **五大基本组件**：
  - **卷积层 (Conv)**：利用卷积核提取局部特征。
    - **感受野与卷积核**：三个 3x3 卷积层（步长为1）的有效感受野等同于单个 7x7 卷积层。采用较小卷积核（如 3x3）能在相同感受野下减少参数、降低计算量并引入更多非线性，从而提升模型性能与泛化能力。
  - **池化层 (Pooling)**：通常使用最大池化，在特定窗口内提取最大值。作用是进行下采样以减少参数量，同时保留最显著特征。
  - **激活函数层 (Activation)**：引入非线性，使网络能学习复杂函数。
  - **归一化层 (Normalization)**：稳定数据分布，加速训练。
  - **全连接层 (FC)**：将池化层输出的多维特征图展平成一维向量，以便输入全连接层。最后计算全连接线性变换、Softmax 概率以及交叉熵损失。
- **Dropout**：
  - **核心含义**：在每次前向传播中，随机将部分神经元置零。即对每个神经元独立地以概率 p 进行保留（以概率 $1-p$ 将其置为 0）。丢失概率是超参数，通常取值为 0.5。
  - **优势与本质**：迫使网络采用冗余表示；阻碍特征间的协同适应。本质上是在训练一个共享参数的大型模型集合。
  - **训练阶段**：每次生成的 mask 是随机的（不保证恰好一半被保留），直接将被丢弃的神经元置零，不进行缩放。
  - **测试阶段**：所有神经元始终处于激活状态（不再丢弃）。为使输出强度与训练期望一致，我们必须调整激活值，使每个神经元满足：测试时输出 = 训练时预期输出（即将激活整体乘以保留率 p）。
- 卷积、池化后矩阵大小的计算公式：$O=\left\lfloor\frac{I+2P-K}{S}\right\rfloor+1$

  参数说明：
  - O：输出特征图的大小（Output size）
  - I：输入特征图的大小（Input size）
  - P：填充大小（Padding）
  - K：卷积核或池化核的大小（Kernel size）
  - S：步长（Stride）

(2) 激活函数

- **核心目标**：为模型引入非线性。
- **ReLU**：
  - 公式为 $a=\max(z,0)$ 或 $RELU(z)=\max(0,z)$。导数规则：在 $z>0$ 时导数为 1，在 $z\leq 0$ 时导数为 0。将所有负值过滤为 0。
  - 优势：在正区域内不饱和；计算效率极高；实际收敛速度远快于 Sigmoid；有效缓解梯度消失问题。
  - 缺陷：输出值非零中心化；当输入小于 0 时会出现死亡 ReLU 现象（即梯度为 0，参数停止更新）。Leaky ReLU 能更好解决此问题。
    - **Leaky ReLU**：ReLU 的变种。当 $z\leq 0$ 时给予一个极小的斜率（如 $0.01z$），避免神经元在负区间陷入“死亡”状态。

- **Sigmoid**：
  - 公式为 $a = \frac{1}{1+e^{-z}}$。导数恒等式为 $\frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y})$。
  - 优势：将数值（线性输出）压缩到 $[0,1]$ 区间，提供非线性表达能力，常用于二分类的输出层；输入为负时仍有梯度，可避免死亡现象。
  - 缺陷：导数最大值仅为 0.25。多层 Sigmoid 激活导致梯度逐渐衰减，极端 z 值导致梯度基本消失，靠近输入层的参数停止更新。
- **GELU**：
  - 公式为 $a = z \times \phi(z)$。
  - 优势：在 0 附近表现优异；平滑特性有助于实际训练过程。
  - 缺陷：计算成本高于 ReLU；大负值仍可能导致梯度趋近于零。

- **Softmax**
- 公式为 $P(Y=k|X=x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}}$。将模型输出的原始得分转换为概率分布。输出必须满足两个条件：所有输出为非负（即在 $[0,1]$ 区间内），并且总和为 1。
- 优势：将无界的原始得分转化为直观的概率分布；非常适合作为多分类任务的输出层，常与交叉熵损失函数结合使用。
- 缺陷：指数运算易放大数值差异，对极端异常值极度敏感，易导致数值溢出（上溢或下溢）；由于概率总和固定为 1，各类别间存在强互斥性，不适用于多标签分类任务。

(3) 卷积神经网络架构

- **VGGNet：小卷积核的胜利**
  - **核心改动**：抛弃大尺寸卷积核，全面采用堆叠的 3×3 **小卷积核**。
  - **优势**：
    1. **参数更少**：2个 3×3（感受野等同于 5×5）或 3个 3×3（感受野等同于 7×7）大大减少了参数量。
    2. **非线性更强**：每次小卷积后都接入 ReLU，多层堆叠带来了更多的非线性变换，显著增强了模型的特征表达能力。
- **ResNet：突破深度的极限**
  - **退化问题 (Degradation Problem)**：当普通网络（Plain Network）堆叠得过深时，训练误差不仅没有降低反而升高，信息在层层传递中丢失。
    - **残差连接 (Residual/Skip Connection)**：引入跨层的跳跃连接，公式为 $H(x) = F(x) + x$。
    - **作用机制**：如果新增的网络层不需要学习任何新特征，它只需将自身的权重推向 0，使 $F(x) = 0$，从而轻松实现**恒等映射 ($H(x) = x$)**。这保证了网络加深时性能绝不会比浅层网络差，成功训练出了上百层的超深网络。

(4) 权重初始化

- **全零初始化的灾难**：导致同一层所有神经元前向输出和反向梯度完全一致，陷入**对称性危机 (Symmetry Breaking)**，整层网络退化为单一神经元的表达能力。
- **极小/极大随机初始化**：分别会导致严重的**梯度消失**（激活值层层衰减趋零）和**梯度爆炸**（激活值层层放大至 NaN）。
- **经典初始化方法 (保持方差稳定)**：
  - **Xavier 初始化**：适合 Tanh / Sigmoid。
  - **Kaiming (He) 初始化**：专为 ReLU 设计。因 ReLU 截断负数导致方差减半，需额外乘 2 补偿能量损失。

(5) 数据预处理

- **核心操作**：对每个通道进行中心化和缩放，减去各通道均值，除以各通道标准差。
- **目的**：使不同维度的特征量级一致，让损失函数的地形更“圆”更均匀，从而防止震荡并加速收敛。

(6) 数据增强

- **手段**：随机裁剪、水平翻转、颜色抖动、平移缩放等。
- **目的**：在不增加标注成本的前提下人为扩充数据集，减少过拟合，显著提升模型的**泛化能力**。

(7) 迁移学习

- **解决痛点**：从头训练深层 CNN 需要海量数据，但在实际任务中往往面临“数据不足，极易过拟合”的困境。
- **核心思想**：利用在庞大数据集（如 ImageNet）上已经训练好的模型，将它们作为通用特征提取器应用到新任务上。
- **策略**：
  - **当目标数据集很小**：只修改最后一层（全连接层）以匹配新任务的类别数。**冻结 (Freeze)** 前面的所有卷积层权重，仅训练最后一层，防止过拟合。
  - **当目标数据集较大**：可以解冻更多后面的卷积层，甚至一起以极小的学习率进行微调，让模型更加适应新领域的数据特征。

(8) 超参数选择

- **科学调试步骤**：
  1. 检查初始 Loss。
  2. **小样本过拟合**：验证代码逻辑是否正确。
  3. 寻找使 Loss 下降的学习率（1e-1 到 1e-5 范围尝试）。
  4. 观察曲线：
     - **训练集与验证集差距巨大**：过拟合（需增加正则化或数据增强）。
     - **两线几乎重合且准确率低**：欠拟合（需增加模型容量或延长训练）。
- **学习率异常**：若 Loss 变为 **NaN**，通常是因为学习率设置得过大。

7. 归一化

- **层归一化(Layer Normalization)**：核心思想是学习能够对输入数据进行缩放/平移的参数。对每个样本（每一行）的所有特征独立进行标准化。
  1. 计算均值：$\mu_i = \frac{1}{d}\sum_{j=1}^{d} X_{i,j}$。
  2. 计算方差：$\sigma_i^2 = \frac{1}{d}\sum_{j=1}^{d}(X_{i,j}-\mu_i)^2$。
  3. 标准化：$\hat{X}_{i,j} = \frac{X_{i,j}-\mu_i}{\sqrt{\sigma_i^2}}$。
  4. 缩放和平移：$y = \gamma(x-\mu)/\sigma + \beta$ 或写为矩阵形式 $Y = \gamma \odot \hat{X} + \beta$。